

Scuola di Ingegneria e Architettura
Corso di Laurea Magistrale in Digital Transformation Management

Observability through a telemetry framework for distributed web apps: error and performance monitoring for Technogym Mywellness CRM

Tesi di laurea in:
BIG DATA AND CLOUD PLATFORMS

Relatore

Prof. Matteo Francia

Candidato

Francesca Gaeta

Correlatore

Dott. Davide Domini

II Sessione di Laurea
Anno Accademico 2025-2026

Abstract

Modern distributed systems and microservices architectures present unprecedented challenges for enterprises in understanding application behavior in production. Traditional monitoring approaches, that rely on known failure modes, are insufficient to face the unpredictability of contemporary digital ecosystems. Imagine monitoring an e-commerce to recognize when the API dedicated to payments is being slower than usual. Basic monitoring can intercept the existence of the issue, but not rebuilding the reasons behind it, which has to be investigated manually. This is the reason why observability has emerged to support digitalized companies in this situation, since it is capable of inferring internal system states from external outputs such as logs, metrics and traces, without any prior knowledge of specific failure scenarios. For the e-commerce, the system allows noticing that the slowdown only involves European users, or that after the latest deployment something changes in the indexes used for the query and this brought a worsening of the service, and it is easier for developers to retrace the steps. This thesis explores the practical implementation of observability frameworks in entrepreneurial contexts, addressing the fundamental shift from reactive troubleshooting to proactive system intelligence evaluating modern observability platforms. Technogym Spa serves as case study for this research. Specifically, the core objective was providing users with high quality digital products and assistance. Before this infrastructure, the customer experiencing a problem while using the CRM services had to report it to the technical assistance. Even though developers were fast in solving the errors respecting the one-day deadline for high criticality, the whole process to make them aware of the alert expanded the time window during which the client could not access the broken parts of the application, lasting even days. They wanted to avoid it to add more value to the customer. The implementation focuses on the Mywellness CRM Progressive Web Application, demonstrating how observability principles can be applied to React-based frontend applications to enable proactive error detection and performance optimization while respecting strict privacy regulations.

*Cadendo, la goccia scava la pietra,
non per la sua forza, ma per la sua costanza.*

Contents

CONTENTS

List of Figures

LIST OF FIGURES

List of Listings

4.1	sanitization function	41
4.2	API error handling	46
4.3	RUM SDK trace context	47
5.1	recordException() method for handled errors	53
5.2	componentDidCatch() implementation for ErrorBoundary	53
5.3	snippet of the beforeSend hook	55

LIST OF LISTINGS

Chapter 1

Introduction

The landscape of enterprises, especially the highly technological ones, or those strictly related to software development, has undergone a radical transformation over the past two decades. Organizations have progressively migrated from monolithic applications, where all functionalities resided within a single codebase, toward distributed architectures composed of numerous microservices dedicated to different business capabilities. This evolution was driven by the need for independent scalability of different application components, the ability to deploy updates without system downtimes, the flexibility to adopt diverse technology stacks optimized for specific tasks, and the organizational agility to enable autonomous teams working on different services simultaneously.

Furthermore, the adoption of cloud computing platforms intervened in decoupling infrastructures to overcome strict hardware constraints and pushing towards horizontal scaling and global distribution. This elasticity introduced a new dimension of operational complexity driven by a multiplied number of components, their interactions, and potential failure points.

All things considered, traditional monitoring paradigms were designed for a simpler era. These systems operate on the principle of *known unknowns*, which means they excel at answering specific, predefined questions, and monitoring tools rely on static dashboards configured on the basis of historical failure modes that

engineers have previously encountered and explicitly instrumented. However, we can spot emergent behaviors that cannot be predicted by examining individual components in isolation. Failure modes became unpredictable and cannot always be anticipated by simple static alerts, entering the era of (*unknown unknowns*). For instance, a performance degradation might be due to the interplay of multiple factors, such as network latency and database query optimization, therefore it comes to help to dispose of a kind of information that lets us walk the path back to explore the plausible chain of problems. Traditional monitoring falls short in this scenario because no one would have thought of those possibilities that generated the error.

The business impact of this monitoring gap is substantial. Apart from causing higher costs for the company, slow page loads, unresponsive interfaces, intermittent errors, erode user trust gradually, and we only discover it not thanks to metrics, but when we observe the abandonment rate increasing. For organizations competing in digital markets, the ability to rapidly diagnose and resolve issues directly translates into competitive advantage.

For a global leader such as Technogym, where the intersection of high-end industrial manufacturing and sophisticated digital ecosystems creates a unique, wide operational environment, this shift is determinant. Through observability, rather than trying to predict any possible scenario - which is basically impossible -, we can focus on ensuring the systems emit rich telemetry data to recognize anomalies a posteriori. The true power of this technique emerges from the correlation of three essential signals that we will better disclose throughout the thesis: metrics, logs and distributed traces. In the examined case study, we will document the implementation of a project aimed at enhancing error monitoring and system performance declined for the corporate Mywellness CRM app. By leveraging the **OTel!** (**OTel!**) framework, alongside AI-driven correlation analysis, we can transform raw telemetry into actionable insights. This shift from reactive troubleshooting to predictive maintenance represents a significant value-add for the Technogym's ecosystem, ensuring that *Wellness* remains a seamless experience across both physical and digital domains.

The research is organized as follows:

- **Chapter 2:** provides an overview of Technogym’s corporate philosophy and internal digital processes, highlighting the cultural drive for constant innovation. We will get acquainted with the project.
- **Chapter 3:** explores the theoretical foundations of *observability*, detailing the three pillars of telemetry (metrics, logs, traces) and focusing on the **OTel!** standard and a comparative analysis of modern monitoring platforms to identify the most suitable solution for the enterprise context.
- **Chapter 4:** documents the practical implementation of the infrastructure from the initial **POC! (POC!)**, an experiment conducted to validate the feasibility, functionality, and potential of a technology, to the integration of custom libraries. We will dive into the code architecture to understand how monitoring is enabled and integrated in the app, plus respect privacy compliance mechanisms including URL sanitization and session replay mechanisms.
- **Chapter 5:** analyzes the project’s primary outcome, a robust error-monitoring system with its associated dashboards for quick-glance analysis, including performance benchmarking, user behavior analysis via heatmaps and session replays. It discusses the technical hurdles, **KPI! (KPI!)** definitions, and the resulting improvements in incident response workflows.
- **Chapter 6:** concludes the thesis with a reflection on the potential future evolution of the project and the professional and personal growth achieved during the internship.

Chapter 2

Technogym: the wellness ecosystem

In Technogym, innovation is not a destination, yet a constant state of motion. A recurring sentiment within the company is that “once a goal is reached, it is already part of the past”. This is the motivation that makes it outperform competitors and positions it as one of the most influential wellness companies in the market, as recently stated by the *TIME Magazine*.

Founded in 1983 by President Nerio Alessandri and headquartered in the Technogym Village in Cesena, the company has consistently redefined the boundaries of the “Wellness” experience. However, in an era where digital touchpoints are as critical as physical equipment, the company’s ambition has shifted to “proactive” excellence. The goal is no longer just to provide a high-quality interface, but to achieve a level of operational transparency where bugs or performance regressions are intercepted before the user even perceives them. Ideally, issues should be identified and resolved before a customer even feels the urge to contact support. The company’s DNA is built on the concept of the *Wellness Valley*, which integrates physical health, mental well-being, and technological innovation. Internally, this translates into a high-precision work environment where automation and digital documentation are paramount.

There are four main words to identify Technogym's drivers:

1. **Technology**, which seems quite trivial considering the world that Technogym has created and the reality we live in today, but in 1989 it was totally visionary. Technogym always led technology-driven changes in the fitness industry, setting high standards for equipment built by other companies. Nowadays, enterprises are considering digital transformation and are always more prepared to face innovation, which might put Technogym in a bad spot, and this is the reason why it always thrives with newer solutions and endless enhancement to protect its relevance and leadership.
2. **Personalization**, what motivates the users to start a training program with the aid of the app through which they can get in touch with professionals based on their requests, routines and, most importantly, unique needs, assigning customized programs for a highly personal experience.
3. **Health**, which is the most emphasized part of the life of people. Technogym pushes it at the vertex of the pyramid, without forgetting other important aspects such as amusement, passion, socialization, strength, and aesthetic purposes.
4. **Ecosystem**, probably the real essence that encloses it all. Technogym was the first to introduce a *digital key* to access its machines with a user account where all personal data and training progresses are collected, to then switch to the modern web applications released either for professional operators in clubs, hotels and hospitals who need to follow their clients, and for common people that want to gain access to a training program in an easy and efficient way to improve their lifestyle and habits while feeling assisted; it was the first to make clever usage of the cloud when it was first developed to really become seamless.

2.1 A React-based implementation framework for observability: the project for the Mywellness CRM web application

Technogym operates as a hybrid of a hardware manufacturer and a software house, thus are we interested in grasping the context around the Mywellness CRM, the application the project was tailored on, to understand its functioning and the weak points the development team wanted to intervene on in order to boost customer experience and accelerate the debugging process.

The Mw-CRM is a **PWA!** (**PWA!**) that enables fitness professionals to manage client relationships, design personalized training programs, track progress, and deliver engagement through web and mobile interfaces. **PWA!** [?] are web applications built and enhanced with modern APIs to provide enhanced capabilities while still reaching any web user on any device with a single codebase. Reliable apps also need to be usable regardless of network connection. Users expect apps to load quickly on slow or flaky network connections, or even when offline. Making the PWA installable moves it out of the browser and into a standalone window. Installed PWAs can be launched on the user's home screen, dock, taskbar, or shelf. The user can search for them on a device and jump between them with the app switcher, making them feel like part of the device they are installed on.

The CRM platform is designed to be used by professionals who support their clients during their training journeys. It should not be confused with the well-known Technogym App, which is instead the mainstream app for very final users who train in the gyms around the world, yet the CRM is the bridge such facilities and their operators have to reach and guide their clients during their workouts and fitness journeys, even by exploiting the Technogym App itself (for instance, messages and updates can be sent directly there). With seamless integration of third-party software, smart equipment, and wearables, Mywellness (??, ??) puts operators in full control of their digital strategy and members engagement. It is statistically measured to increase attraction of 40% and retention of 30% thanks

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION

to variety, customization and automation. More than 30 thousand facilities and 40 million users are registered on the platform, with more than 500 thousand workouts tracked. The application is extremely complex due to its level of detail in covering a huge number of cases and functionalities. The workflow it follows consists of an ensemble of subsequent steps, starting from collecting health and fitness information to build the user’s health history, then offering direct booking of courses/trainers and payment possibilities, defining the assessment to discover the user’s goals and consequently building the best personalized training program, consistently tracking progress, offering communication channels through which coaches can stay in touch to motivate clients, creating challenges to encourage friendly competition and keep users engaged, and finally rewarding results and loyalty. The Mw-CRM requires frontend programming expertise, but something fascinating about the observability project itself was that it could be interpreted as a “backend solution for the frontend”, involving a combination of both, even without directly intervening on BE components as APIs or databases.

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION

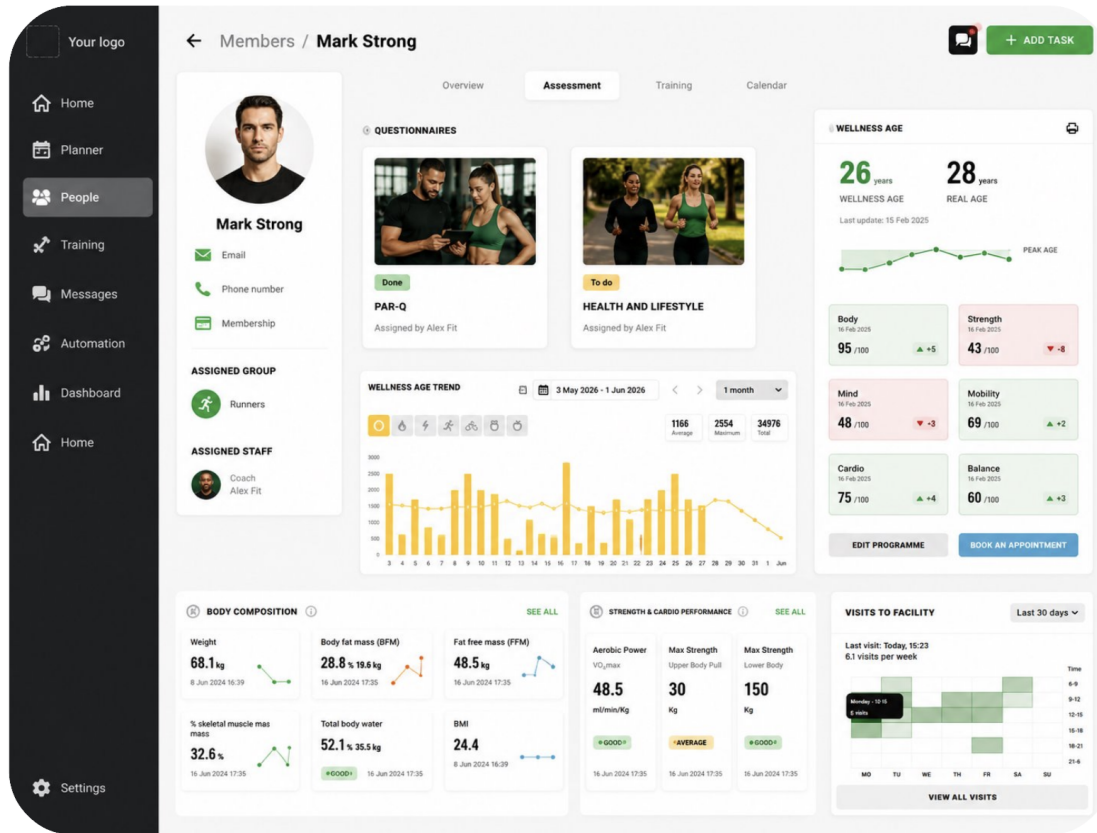


Figure 2.1: CRM Web App: People Section

All the services, platforms and processes mentioned in broad terms in these sections underline the transition trend from traditional manufacturing to a digital-first approach that already began years ago. The frontend team continuously works to redesign the interface when it can be elevated, listens to and satisfies customers' requests and suggestions that would make their sessions in the gym better and, of course, it must solve bugs and errors. It is fundamental to put the accent on these tasks to concretize the "The Present is Already the Past" mindset, possibly linked to the need for observability: to stay ahead of the market, the infrastructure must be self-aware and constantly evolving. One of the best ways to do so is to introduce tools and tune AI systems to automate and speed up problem resolution, while acquiring insights about what has changed with respect to previous versions, or years, or functionalities. The right data to nourish this whole workflow can

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION



Figure 2.2: CRM Web App: Class Scheduling Section

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION

be collected by exploiting observability principles, that we are going to officially disclose in the next chapter as focal point of the paper.

In relation to what we illustrated until now, the request Technogym advanced was to create a complete and reusable library for **React** and **TypeScript** applications that enables application observability management through a single integrated platform. The library must collect and transmit telemetry data using the OpenTelemetry protocol to monitor errors, performance, user interactions, and usage statistics. The reason for implementing a library is that, ideally, observability could be considered exclusively a CRM goal at first, to then be expanded to other digital products just by importing it.

As main features, we find:

- **Error Monitoring:** automatic capture of JavaScript errors, React error boundaries, API calls, complete stack trace tracking, handling of asynchronous errors and promise rejections, error categorization and prioritization. This initial phase covers the main deliverables the company committed to.
- **Performance Monitoring:** Core Web Vitals measurement (LCP, FID, CLS), page and resource loading time tracking, React component rendering performance monitoring, API call response time analysis, threshold definition for sending alerts via email and Slack notifications, a cloud-based team communication and collaboration platform to centralize files and messages;
- **Heatmaps and Scrollmaps:** recording of user click positions, heatmap generation to visualize the most interacted areas, page scroll depth tracking, navigation and interaction pattern analysis. All these elements contribute to giving new ideas about possible enhancements of the design of the UI to meet user expectations regarding usability and responsiveness;
- **Usage Statistics:** collection of browser information, version, and user agent, user geolocation (country, region), device data (desktop, mobile, tablet) and screen resolution, engagement metrics (time on page, bounce rate), tracking of visited pages and navigation paths, event tracking to evaluate feature usage.

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION

The technology stack is expected to include version 18+ of React and 5+ of TypeScript for the frontend, **OTLP!** (**OTLP!**) as protocol to acquire inherent data, Vite as build tool for library bundle creation, Vitest for testing and Vite Plus with OXLint for linting and formatting, which concerns the process of running a program that will analyze code for potential errors. Linting focuses on enforcing coding conventions and best practices, detecting syntax errors and improving code readability as a sort of static analysis to catch issues early to maintain code consistency and clarity. Testing, whose debugging is a synonym, involves manually fixing the code after a bug has been found, generally after the program has run, and this differentiates it from the linting practice. This latter, before was implemented through the *ESLint* library for the validation part, and the *prettier* library for the formatting part, and thanks to plug-ins they could function simultaneously. Now, they switched to *Vite Plus* with *OXLint* because claimed to be up to 40 times faster and more powerful than traditional systems. Integrating a single package instead of keeping both ESLint and Prettier will reduce configuration conflicts and dependency vulnerabilities.

The workflow is intended as follows:

- **Phase 1.** In depth analysis of OpenTelemetry and its APIs, study of existing libraries and backend platforms to define the architectures to implement.
- **Phase 2.** Core implementation (with precedence to error tracking) to achieve the goals and observe the requirements mentioned above, adding the development of precise unit tests to simulate errors to check if they are collected in the platform. The idea is to start from a blank template that will serve as demo app and will be modified according to our needs to test if what we are building works before migrating everything to the real application, while structuring the related documentation. Following the theoretical analysis from phase 1, a practical comparison was conducted between Click-Stack and Datadog, the most suitable candidates identified, to determine the optimal platform for the project requirements.
- **Phase 3.** Configuration of dashboards to monitor errors in the platform, performance statistics especially related to APIs health, and of alerting sys-

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR OBSERVABILITY: THE PROJECT FOR THE MYWELLNESS CRM WEB APPLICATION

tems to notify errors from the platform in real time. Activation of session replay and user events recordings.

- **Phase 4.** Merge the observability implementation from the ad-hoc feature branch into the beta environment branch for a pilot deployment. This trial phase serves three objectives: validating telemetry collection with real user traffic, iteratively refining the implementation based on observed issues if any, and projecting production costs to determine the scope of the contract in the future.
- **Expected Deliverables.** A library source code published on Git repository, npm package ready for distribution, a complete technical documentation, and a real-time monitoring system with integrated threshold- and criticality-based notifications, adding an overall final presentation to show processes and results.

Each step will be further analyzed and understood as the reading proceeds in the designated parts.

2.1. A REACT-BASED IMPLEMENTATION FRAMEWORK FOR
OBSERVABILITY: THE PROJECT FOR THE
MYWELLNESS CRM WEB APPLICATION

Chapter 3

Observability foundations and platform architectures: from OpenTelemetry standards to enterprise solution selection

Historically, system health was managed through *monitoring*, a reactive process centered around “known unknowns”. Monitoring tools are designed to answer specific questions, like discovering if the server is up, or the CPU usage percentage. It is centered around events that people could predict in advance. Modern software architectures require more than just monitoring; they require *observability* [?], the ability to understand a system’s internal state solely from its external outputs (logs, metrics, and traces). Technogym needed this, as it transitioned into a distributed system of microservices and cloud-integrated fitness equipment, and failure modes became unpredictable and could not be anticipated by simple static alerts (*unknown unknowns*).

In an observable system, we do not just wait for a threshold to be crossed; we collect enough granular data to understand why something is happening for issues we have never experienced before. For the Mywellness CRM app, this means the

difference between knowing the app is slow and knowing that a specific API call is lagging for users on a specific firmware version in a specific region. Capturing various severity level types for logs (e.g., error, info, warn, ...); associating types of data with personalized message bodies to understand in natural language what crashed; building dashboards and reports; these all embody strategies to speed up the correction of bugs and malfunctions.

To achieve this level of insight, the industry relies on three fundamental types of telemetry data that we wanted indeed to intercept during the project [?]:

- **Metrics.** Metrics are numerical representations of data measured over intervals of time. They are incredibly efficient for storage and ideal for spotting trends or triggering alerts. In the Technogym ecosystem, metrics might track the number of active workouts per second or the memory consumption of a **CRM! (CRM!)** background process. They cause low overhead and retention is set to the long-term, but they are not enough alone to understand the context, since they just warn when *something* is wrong, but not *where* or *why*.
- **Logs.** Logs are timestamped text records of discrete events. They provide the report of a failure. Whether it is a 404 error on a login page or a database connection timeout, logs offer the specific details needed for debugging. They are pricier to store and difficult to search at scale without proper indexing. It is not compulsorily associated to a predefined user, this is the reason why it should be correlated to other measurements.
- **Traces.** Distributed Tracing is arguably the most critical pillar for modern apps like Mywellness. A trace follows a single request as it travels through various services (e.g., from the user's smartphone to the authentication layer, then to the database, and back). It provides a map of the journey, highlighting exactly where delays or errors occur. Traces show the exact path and timing of service calls, reveal dependencies between services, highlight performance bottlenecks, and provide context about each step in a request's life cycle, this is why traces and logs become extremely powerful if coupled up.

There are other kinds of information we can acquire thanks to **spans**, that are single units of work and track specific operations that a request makes (actually a trace can be composed of one or more spans), and there is a whole table of attributes available to cluster spans, or **events** as structured records of significant occurrences in the system, or **profiles** that provide deep analysis of resource consumption and code execution patterns, but the ones in the paragraph above are the most important ones which automatically link to these various subcategories.

3.1 The role of OpenTelemetry: vendor-neutral standardization and the collector architecture

OpenTelemetry [?] has emerged as the industry's answer by offering a unified framework for collecting and managing observability data. It focuses on the collection, processing, and export of telemetry data, leaving the monitoring and alerting aspects to other observability tools. Its strength is being an open-source and vendor-neutral framework that standardizes how we collect and handle telemetry data across distributed systems.

The vendor neutrality approach extinguishes a major problem in modern observability: the fragmentation of telemetry collection methods. Instead of maintaining a different instrumentation code for each monitoring tool, we can instrument with **OTel!** the first time and send data anywhere, instead of managing different pipelines for logs, metrics, and traces. It acts like a common language for observability data that works across all services, regardless of the programming language. We must be aware that this library is not a monitoring or alerting tool itself, yet it provides the means to collect telemetry data from applications without built-in monitoring capabilities that we should tune externally.

Other important aspects are the preservation of backward compatibility with existing implementations. Before, vendor dependency led to using fragmented

3.1. THE ROLE OF OPENTELEMETRY: VENDOR-NEUTRAL STANDARDIZATION AND THE COLLECTOR ARCHITECTURE

tools with different languages and proprietary agents and high maintenance overhead, but with OpenTelemetry everything is unified and language-agnostic. **OTel!** uses the **OTLP!** to move data between components. The hint from the previous considerations is that the library does not immediately offer a storage, but we can agglomerate to our architecture any type of instrument, like the *OTel Collector* in a natural way that acts as a central hub [?]. It takes in telemetry from different sources (not just OpenTelemetry SDKs, but also Zipkin, Prometheus etc.), processes it, and sends it wherever, which means we do not need to hard-wire the application to any backend. The collector is in charge of routing, formatting and cleaning up, and it can run in different modes: in *agent mode* it runs on the same machine as the app and filters or enriches data before sending it out; in *gateway mode* it runs as a central service that collects telemetry from multiple apps or instances. Nothing prohibits to use both, therefore collect locally and then forward centrally.

The collector (??) works as a modular pipeline with four key components:

- Receivers that handle incoming data from various protocols
- Processors that modify, filter, or batch data
- Exporters that send processed data to different backends
- Connectors that pass data between pipelines or generate new telemetry types

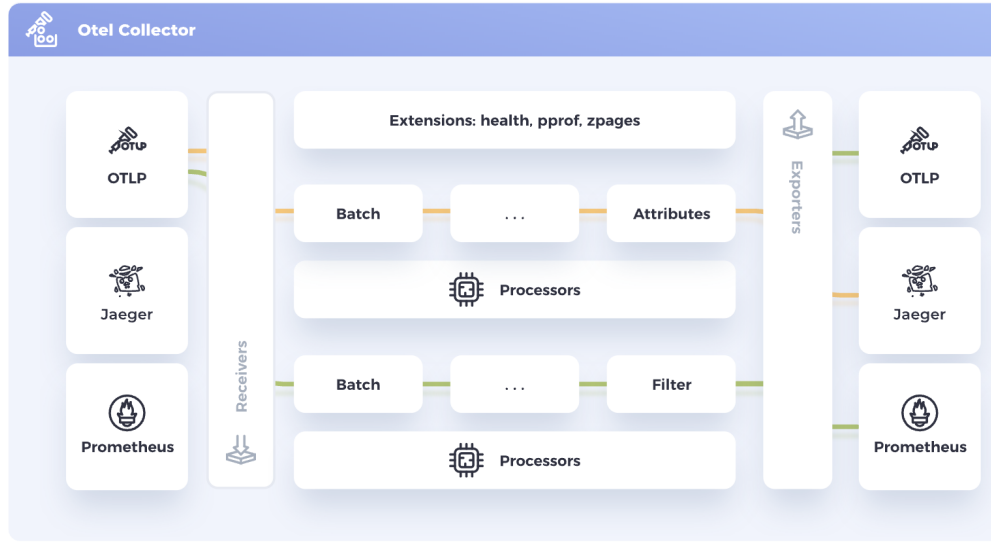


Figure 3.1: OpenTelemetry Collector Architecture.

Source: <https://opentelemetry.io/docs/collector>

For getting started with OpenTelemetry, sending data directly to a backend is a great way to get value quickly. Also, in a development or small-scale environment - which is not our case -, we can get decent results without a collector. However, it is recommended using a collector, since it allows the service to offload data quickly and the collector can take care of additional handling like retries, batching, encryption or even sensitive data filtering. Since standardization of movements does not tie to any vendor, if we need to switch tool we just update the collector's configuration in one line.

3.2 Possible limitations in OpenTelemetry

While OpenTelemetry provides a robust framework for digital transformation, its implementation is not exempt from significant trade-offs [?]. For a complex ecosystem like Technogym's, several inherent limitations must be addressed.

The act of observing a system inevitably consumes resources. Instrumenting applications with OTel SDKs introduces CPU and memory overhead, as the application must now generate, buffer, and export telemetry data. In high-throughput environments, such as processing thousands of simultaneous workout heart-rate samples, the “observability tax” can impact the very latency we aim to measure. This requires careful tuning of sampling rates to balance data depth with system performance.

Observability shifts the burden from “not having enough data” to “having too much”. Traces and logs, especially when highly granular, generate massive volumes of data. Storing and indexing this information in backends can lead to skyrocketing infrastructure costs. Without a clear sampling strategy where only a percentage of successful traces are kept while 100% of errors are captured, the project risks to become financially unsustainable.

Simply collecting the three pillars is insufficient; the true value lies in their **correlation**. If a log entry cannot be linked to a specific trace ID, or a metric spike cannot be drilled down into a trace, the data remains siloed. Ensuring that all microservices correctly propagate context across different programming languages and protocols is a significant engineering challenge that requires strict organizational standards.

It is critical to reiterate that OpenTelemetry is a collection standard, not a storage or analysis solution. While OTel unifies how data is sent, it does not provide the *single pane of glass* for viewing it. Technogym must still maintain and manage backend databases and visualization layers. Furthermore, as **OTel!** is a relatively young and rapidly evolving project, certain components (specifically the logging specification and profiling) are less mature than tracing and metrics, potentially leading to breaking changes during future updates.

Finally, the transition to observability is as much a cultural shift as a technical one. Engineers must move away from “dashboard-watching” toward “exploratory debugging”. This requires a deeper understanding of the system architecture and a new set of skills in querying high-cardinality data, which may initially slow the

development speed during the adoption phase.

3.3 Platform Architectures: evolution from On-Premise to Cloud

OTel! is excellent for standardized telemetry collection. However, the requirements in our project include **RUM!** (**RUM!**) and **UX!** (**UX!**) analytics features that are not universally covered by OTel-compatible vendors, especially heatmaps and scrollmaps. In practice, most teams end up choosing:

1. an observability backend that supports **OTel!** well, and
2. optionally a UX/session analytics tool for dashboards and heatmaps, due to the fact that many **APM!** (**APM!**) vendors do not do it specifically.

Today, OTel can correctly cover errors, performance and cross-signal correlation, but it does not natively provide heatmaps and scrollmaps, visual **DOM!** (**DOM!**) snapshots, product analytics funnels, retention and cohorts. We can add it thanks to other vendors. Hence, it is essential to identify the critical aspects and distinctions among typologies of platforms to effectively assess the solutions currently available on the market.

The point of interest is that, from the very beginning, data management was centered around the *Relational Data Model*. These systems are designed for schema-on-write, meaning the data structure must be strictly defined before any information is ingested. Also, primordial databases were optimized for **OLTP!** (**OLTP!**), therefore daily operational tasks (e.g., updating a user's weight in a **CRM!**).

3.3. PLATFORM ARCHITECTURES: EVOLUTION FROM ON-PREMISE TO CLOUD

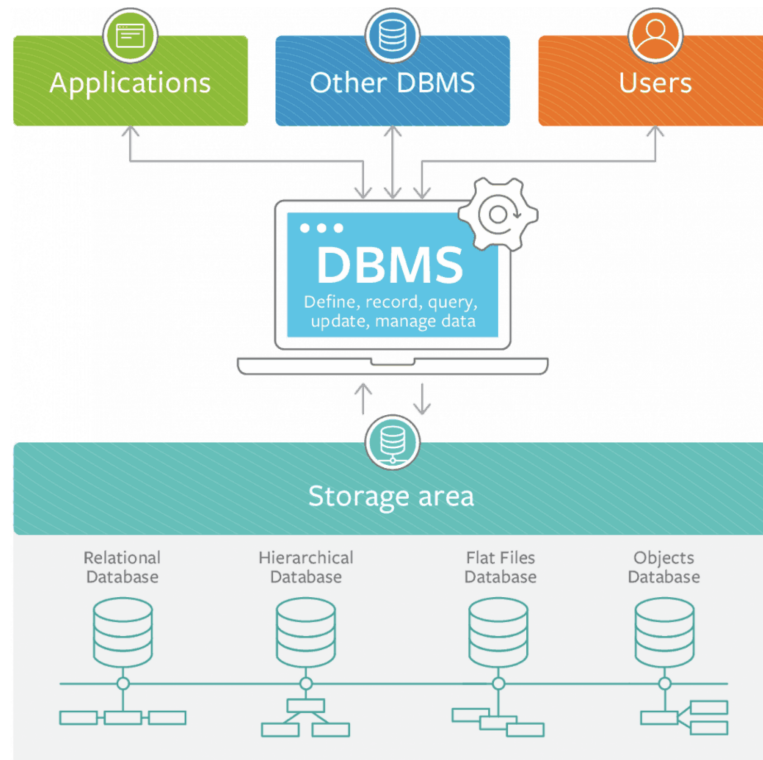


Figure 3.2: Initial Database Management System Architecture.

Source: <https://docs.chaossearch.io>

They excel at maintaining data integrity through **ACID!** (**ACID!**) compliance, but struggle with complex analytical queries across massive datasets. As **BI!** (**BI!**) became critical, organizations transitioned to *Data Warehouses*. These systems separate **OLAP!** (**OLAP!**) tasks from operational ones to avoid performance degradation. While powerful, warehouses remain expensive and rigid, as they require data to be cleaned and transformed via **ETL!** (**ETL!**) before it can be stored.

3.3. PLATFORM ARCHITECTURES: EVOLUTION FROM ON-PREMISE TO CLOUD

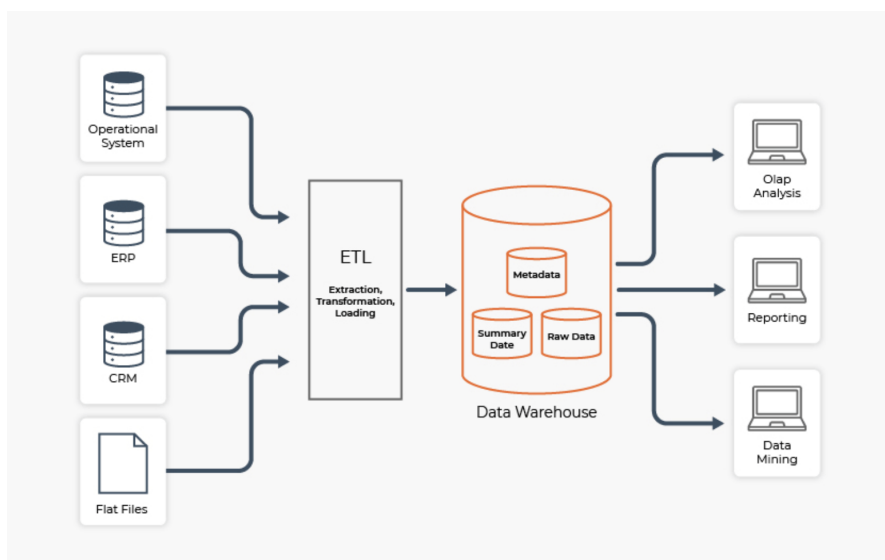


Figure 3.3: Data Warehouse Architecture.

Source: <https://docs.chaossearch.io>

The explosion of *Big Data* - characterized by high volume, velocity, and variety - led to the adoption of the *Data Lake*. Unlike the warehouse, a lake serves as a central repository for raw data. This reflects a schema-on-read, because data is ingested without a predefined schema, avoiding the latency of upfront processing. Anyway, the risk is producing a *Data Swamp*, because it can make data dirty due to the lack of structure even though it is highly flexible and cost-effective for storing massive telemetry logs. Consequently, the most recent architectural wave, formalized around 2021, is the *Data Lakehouse*. This paradigm seeks to eliminate the inefficient “two-tier” architecture where data is duplicated between a lake (for Data Science) and a warehouse (for BI).

3.3. PLATFORM ARCHITECTURES: EVOLUTION FROM ON-PREMISE TO CLOUD

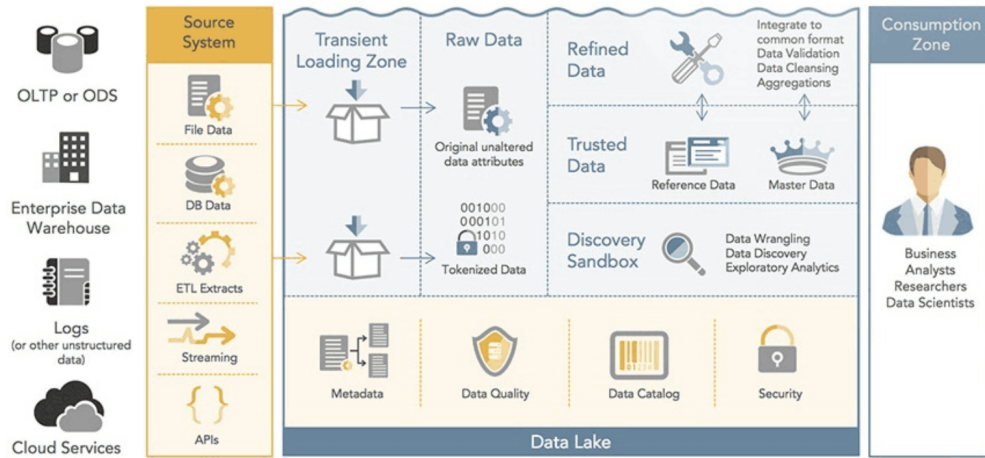


Figure 3.4: DataLake Architecture.

Source: <https://docs.chaossearch.io>

The evolution from relational databases to Lakehouses was not merely a change in data structures; it was a fundamental shift in infrastructure philosophy. Traditional on-premise databases were vertically scaled, meaning that to handle more data, one had to buy a bigger, more expensive server. In the context of Technology’s global expansion, this model became a bottleneck. The transition to cloud platforms (e.g., AWS, Azure, Google Cloud) provided the necessary substrate for modern observability to function. The secret of why cloud platforms work for observability is the decoupling of storage and computation. In traditional DBs, if we wanted to store 10 years of logs (storage), we also had to pay for the CPU (computation) to keep that server running 24/7. In the cloud we can store petabytes of *cold* telemetry data in cheap object storage and only spin up powerful CPUs when we actually need to run a query or investigate a system failure.

The shift to the cloud was driven by three main factors that differentiate it from the old “Server in a room” model:

- Horizontal scalability: unlike a physical database, cloud platforms can distribute a single query across hundreds of virtual machines. This is vital for

searching through billions of Mywellness app traces in seconds.

- Managed services (**PaaS!** (**PaaS!**)): instead of engineers spending weeks configuring database backups and security patches, the cloud provider offers these as a service. This allows the team to focus on analyzing the telemetry rather than maintaining the pipe through which it flows.
- Global distribution: since fitness equipment is used worldwide, cloud platforms allow us to ingest data at edge locations close to the user, reducing latency and ensuring that observability does not slow down the user experience.

Ultimately, the cloud transformed the database into an Ecosystem. We no longer talk about a single tool, but an interconnected platform where eventually OpenTelemetry can send data to a collector, which routes it to a cloud-based Lakehouse, where it is finally visualized by an APM tool. The cloud is the “glue” that allows these modular components to communicate in real-time. Following this wave, Technogym already relied on AWS for storing the enterprise’s data, in addition to Datadog used by the backend teams mainly for its storage capabilities, but actually contains loads of other useful features that suit the case study.

Platforms like Datadog lead the attention to another characteristic that is worth mentioning before delving into platforms analysis: levels of outsourcing on the Cloud and, inherently, the principal vendors and possible architectures to adopt, that are the questions inspiring the research. Since the cloud is not a silver bullet, the decision to adopt it is not binary. The **XaaS!** (**XaaS!**) framework was born to allow organizations to choose their level of responsibility, effectively deciding which layers of the technology stack to manage internally and which to outsource to a vendor like AWS or Datadog. It goes without saying that this specific part will be useful for the considerations that inspire the platform research. It is helpful to distinguish some famous acronyms:

- **On-premise:** the traditional model where we would own the physical servers, networking, and storage in a local data center. It’s best for highly regulated

industries with extreme data sovereignty requirements or legacy systems that cannot be moved. The company would be in total control over the entire stack and data security.

- **IaaS! (IaaS!):** the vendor provides the virtualized hardware (servers, storage, networking), but we manage the Operating System, middle-ware, and applications. It works for organizations that need high customization of the **OS! (OS!)** or want to “lift and shift” existing apps to the cloud without redesigning them, eliminating hardware costs and scaling instantly. We are still responsible for security patches, **OS!** updates and runtime management.
- **PaaS!:** the vendor manages the hardware and the software framework (OS, middle-ware, runtime). We only manage the code and the data. It aligns with developers who want to build and deploy apps quickly due to high development velocity and lower operational overhead. Thus, because of the more predominant vendor lock-in, it can be tougher to move the code to a different provider if we use specific PaaS features.
- **FaaS! (FaaS!):** also known as serverless, a subset of PaaS where we deploy individual snippets of code (functions) that trigger on specific events (e.g., an OTel collector receiving a specific trace). It is the right choice for microservices or intermittent tasks that don’t need a server running 24/7 and gifts extreme cost-efficiency since we pay only at code execution, but it also brings limited execution time and there is a bit of latency when the function wakes up.
- **SaaS! (SaaS!):** the vendor provides a complete, ready-to-use application. We manage nothing but the configuration and the data we feed into it. It is addressed to specialized business needs like observability, CRM, or email where building a custom tool adds no competitive advantage. The advantages are zero maintenance, immediate access to advanced features, but it is the most expensive model at scale, it leaves the least amount of control over how the software works and pushes vendor lock-in to the extreme.

Another important concept to underline is *self-hosting*, that can be intertwined

with that of on-prem. The latter refers to the physical location of the hardware, while the former puts the accent on the act of managing software ourselves instead of using SaaS. The twist is that we can self-host in the cloud by renting a blank **VM!** (**VM!**) on AWS (**IaaS!**) and manually installing a database or an OTel collector on it without being clearly on-premise, because the server is the Amazon datacenter. These subtle questions have been discussed as they represent the holy grail to become familiar with the next steps.

3.4 Platform selection: comparative analysis, requirements and architectural trade-offs

We can put the observability backend and the UX tool together in different ways. The realistic architectural solutions are:

- **One-fit-all:** includes RUM, session replay, alerts, backend observability, and also supports OTel/OTLP export reasonably well.
- **Best-of-breed:** OTel-native backend for errors/performance/trace metrics and a dedicated UX/session analytics tool, stitched together via shared tags as *session.id*, *user.id*, *page.url*, *etc.*

3.4.1 One-fit-all

It means entrusting everything to a unique platform, which would ensure workflow simplicity and linearity and less burden for components implementation. As already hinted, the best framed platforms were Datadog, New Relic and Sentry, which are about to be analyzed to understand the reasoning behind the process.

From the official websites and related documentation, we can learn the analogies among the three that make them interchangeable [?, ?, ?]:

3.4. PLATFORM SELECTION: COMPARATIVE ANALYSIS, REQUIREMENTS AND ARCHITECTURAL TRADE-OFFS

- The possibility of exploiting session replay functionalities with masking techniques to respect privacy requests;
- The access to an EU data center (in Germany, Frankfurt), which reinforces the observance of specific legislation (**GDPR!** (**GDPR!**), AI Act, public DPA and consent and cookies policies for each);
- The usability of an integrated AI, even if some relevant differences will be fathomed in the following;
- Recognition of JS errors, React Error Boundaries, **CWV!** (**CWV!**) for RUM analytics useful for marketing strategies;
- API call response time analysis for the frontend;
- Threshold definition and notification to automatically monitor anomalies, **SLI!** (**SLI!**), **SLO!** (**SLO!**), but it is a built-in functionality only in Datadog and New Relic, while Sentry needs support from other platforms;
- Geo-localization behind consent and GDPR best practices;
- Chance to add SDKs dedicated to full RUM, and these vendors offer their backend storage, which causes a higher vendor lock-in risk and less data portability (the limit is softened in Sentry's case thanks to the telemetry wrapper which introduces abstraction levels);
- They are not self-hosted, but **SaaS!**, further reason of vendor lock-in (Sentry could optionally be adopted as self-host, but this is not the conventional choice, and it is tougher to apply);
- All are OTel-supportive, but Sentry to a lesser extent;
- Data retention period can vary on behalf of the plan and can be customized, but the default is a range of thirty days;
- Access control implementation, mainly for replays, generally adopting **RBAC!** (**RBAC!**) or allow-list in Sentry.

However, we can also list the main differences among the three with a focus on Datadog, since it is the most complete. As a matter of fact, it is full-stack, that means we can save and correlate all signals emitted via native libraries in Datadog or via OTel SDK with OTLP in a single place. The classic data ingestion path can directly exploit Datadog's Agents or use an integrated cloud. Concerning the RUM path, we should use other SDKs to get complete analyses that the exclusive usage of OTel would not be capable of delivering. New Relic is the most complete platform after Datadog, but it is weaker on the UX analysis side with heatmaps, and Datadog is slightly better in providing OTel-compatibility. Here, we can exploit the New Relic AI in addition to a **SRE!** (**SRE!**) Agent, but mostly to assist the user in error analysis and suggest remedies; therefore, it does not seem as powerful as Datadog's combination of Watchdog, for built-in insights, passive and automatic type of research, and bits AI SRE, which is added to the plan with additional fees but can conduct autonomous investigations, which is the extra value we wish to have. In Sentry, instead, the greatest lack is that OTLP insertion supports only traces and logs, not metrics and infra, so we should add a backend platform, which makes everything less doable. Furthermore, it does not directly build heatmaps and is less powerful for marketing metrics, only approximated through spans and replays. In fact, it is an excellent platform on the code and debugging side, which is a central but not exclusive part of our work. Another limitation in these terms is Seer AI, which is development-focused as debugging agent, which means it is not the best solution for our case. The cost model in Datadog is per host and follows ingestion and retention volumes, adding add-ons if any. The only critique against Datadog could be the risk of finding inflated costs at the end of the month. The model could be more transparent, so it requires more attention. New Relic and Sentry are safer, as they respectively follow an ingest-based and user model the former and an events-based and replay volume the latter.

Datadog could be used following a Datadog-first approach to reduce external work, accelerating time-to-value, getting all the possible features, and correlations between frontend and backend. However, it requires more implementation effort to choose the RUM product (it could be Datadog itself using the SDKs or other

specialized platforms like LogRocket or PostHog, which would be safer to have heatmaps and scrollmaps, too), manual correlation among tools through identifiers, alerting logic for the AI to intervene, and the storage in case we do not want to strongly depend on Datadog. OTel is still recommended for vendor-neutrality.

3.4.2 Best-of-breed

Other than what said above involving Datadog, a common solution is the Grafana stack: *Grafana* is the UI layer with dashboards, exploration, alerts, but it does not record telemetry, so it needs a backend beneath. It must be coupled with specialized components in interception of signals, like *Tempo* for traces/spans, *Loki* for logs, *Prometheus* or *Mimir* for metrics. The **OTLP!** in this context is feasible if adopting the collector to receive telemetry from the app, but the backend storage is detached. Interoperability is excellent, and this structure brings cost and control advantages, but pipelines and tools multiply. The only significant advantage with Grafana is making portable elements like dashboards and maps. The best-of-breed has never been fully considered because we did not want to complicate the system this much. Technogym already relies on SaaS, so the team was not worried about keeping this methodology. A percentage of vendor lock-in is welcomed if it keeps organization simpler. It was convenient because Technogym was already a Datadog's client [?].

3.4.3 ClickHouse and the product ClickStack

ClickHouse itself is a high-performance, column-oriented SQL database management system (DBMS) designed for online analytical processing (OLAP). It is available as both open-source and cloud-based offerings, optimized for running analytical queries over massive datasets in real-time. ClickStack, on the other hand, is the ClickHouse observability stack, an open-source platform built in collaboration with HyperDX [?, ?]. It is composed of three blocks: OTel collector (ClickStack distro), ClickHouse storage and its query system (with possibility to use something

3.4. PLATFORM SELECTION: COMPARATIVE ANALYSIS, REQUIREMENTS AND ARCHITECTURAL TRADE-OFFS

else), and ClickStack UI offered by HyperDX. ClickStack as a product can be used in two different configurations: as self-host so that we can manage it all, and a managed ClickStack embracing the SaaS approach, because we should manage the collector or any instrument we decide to use to send OTLP, ClickHouse and HyperDX are hosted and operated in ClickHouse cloud, and it is thought to scale and diminish the burden.

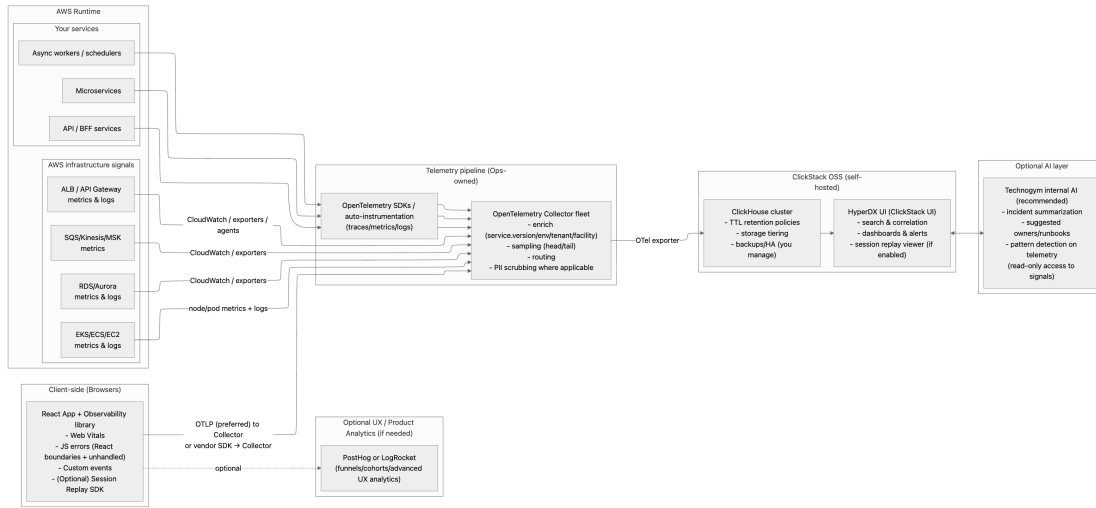


Figure 3.5: Open-source ClickStack Architecture

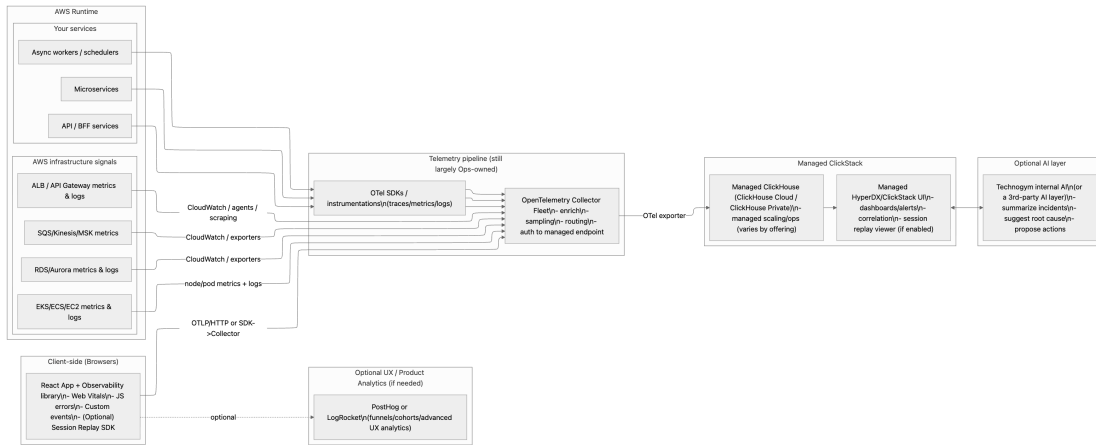


Figure 3.6: Managed ClickStack Architecture

Technologically speaking, the two pipelines are significantly similar; opera-

tionally speaking, we skip the riskiest part in terms of **MTTR!** (**MTTR!**), because ClickHouse (or a general storage) is always on, we acquire the UI, authorization integrations and, most importantly, scaling.

Even though the cost-model is incredibly tempting with ClickStack and ClickHouse (infra-based with promise of storage for less than \$0.03/GB/month and without fee per host), it brings some downsides with respect to Datadog. To begin with, the AI is agentic, but only in the managed version and with the grant of a *private preview* kind of account, hence owners decide who to leave the functionality to, and it is still work in progress, meaning we could experience changes in usage. The UI is not as developed as in Datadog, it offers many fair possibilities, but does not have heatmaps, and it is based mainly on static thresholds, so to access dynamic alerts we should go for two integrated tools into HyperDX, that are *Event Deltas* to analyze traces, and *Event Patterns* to cluster similar log/traces. Plus, there is no explicit reference to **CWV!** capture.

All these motives led us to confirming Datadog as the best solution, affirming again itself as probable leader on the market.

Chapter 4

Telemetry library implementation: from proof of concept to privacy-compliant production integration

In this chapter, we are going to dive into the practical implementation of the project.

4.1 Preliminary Test of ClickStack and Datadog

The operational plan was generating the general telemetry setup to determine the structure and information we wanted any telemetry service to have, enabling the specific configuration for passing and managing telemetry through ClickStack and Datadog, and for completion improvising a demo page as interactive playground to test and verify telemetry collection from the simulated React application. Thanks to few simple buttons, it allowed to manually trigger various telemetry events and see them appear in real-time in the ClickStack UI. The sections are:

- User context to set user identity for tracking user-specific telemetry (user id, email), but for privacy policy it should be masked or hashed, especially when checking session replays; otherwise, the risk is to go against the principles of the GDPR. In ClickStack’s settings it is already possible to establish masking rules for sensitive data;
- Global attributes to add app-wide attributes that appear in all telemetry events (region, app version, browser) to reduce code duplication because it appears everywhere once set, enables filtering and grouping in the UI, and it is useful to correlate events across features;
- Logging (of types *info* for general information message, *warn* for something unexpected but recoverable, *debug* for detailed debugging information usually disabled in production) and error tracking to record application events and messages at different security levels;
- Metrics to record quantitative measurements (counters, gauges, histograms) and simulation scenarios for realistic user workflows with multiple telemetry events at once (e.g., a user journey which simulates a complete e-commerce flow from searching the product to successful purchase);
- User sign up at first account creation;

As already hinted, we tested ClickStack to discover HyperDX’s functionalities. We enabled the access to the local OSS version to avoid creating an account with a limited free trial time. With a ClickStack Docker VM instance, we could explore the interface freely, even though probably the experience in this way was not the same as the one we would have had via registration, and not all features were correctly accessible (such as the AI), but it was enough to grasp the general functioning idea. We formed our environment thanks to the ingestion API key to establish the direct connection with HyperDX, and configured the ClickStack local host endpoint. We used the standard ClickHouse ports to collect and pass telemetry. It already came with automatic browser instrumentation with fetch API calls tracked automatically, XML HTTP requests monitoring and user interactions.

4.1. PRELIMINARY TEST OF CLICKSTACK AND DATADOG



Telemetry Demo Page

Test telemetry collection and verify data is flowing to ClickStack/Datadog

User Context

User ID:

User Email:

Set User ContextClear User

Logging

Basic Logs

Info LogWarning LogDebug Log

Error Tracking

Intentionally trigger different types of errors

Simple ErrorTypeErrorAsync Error

Detailed Error

Figure 4.1: Demo App interface for reference

4.1. PRELIMINARY TEST OF CLICKSTACK AND DATADOG

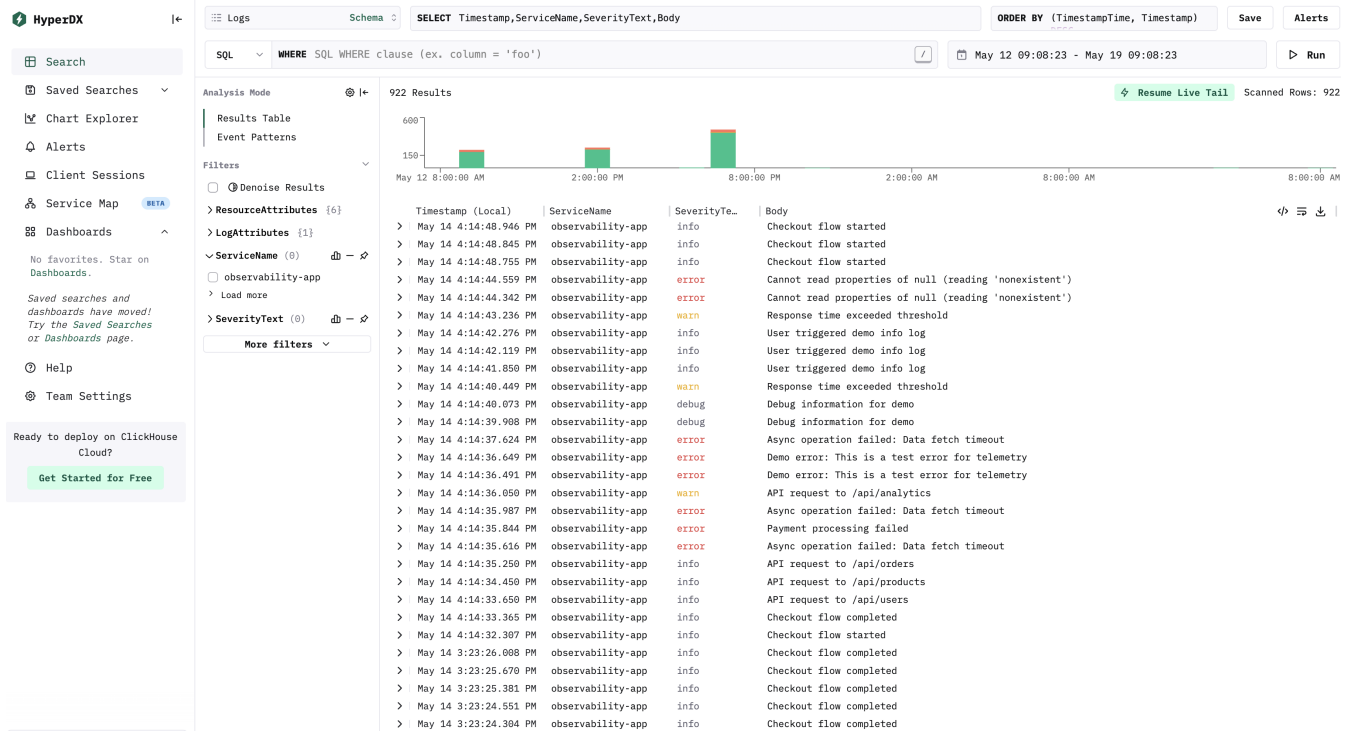


Figure 4.2: HyperDX interface: home page

As we can see in this image (??), thanks to the demo application, we have tested the collection of logs and traces in HyperDX, so that we could see the exact movements of the cursor of the user on the screen. By clicking on a specific row, we can visualize more detailed information in the form of tags that we used to write the SQL queries to build the dashboards. Some examples are:

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

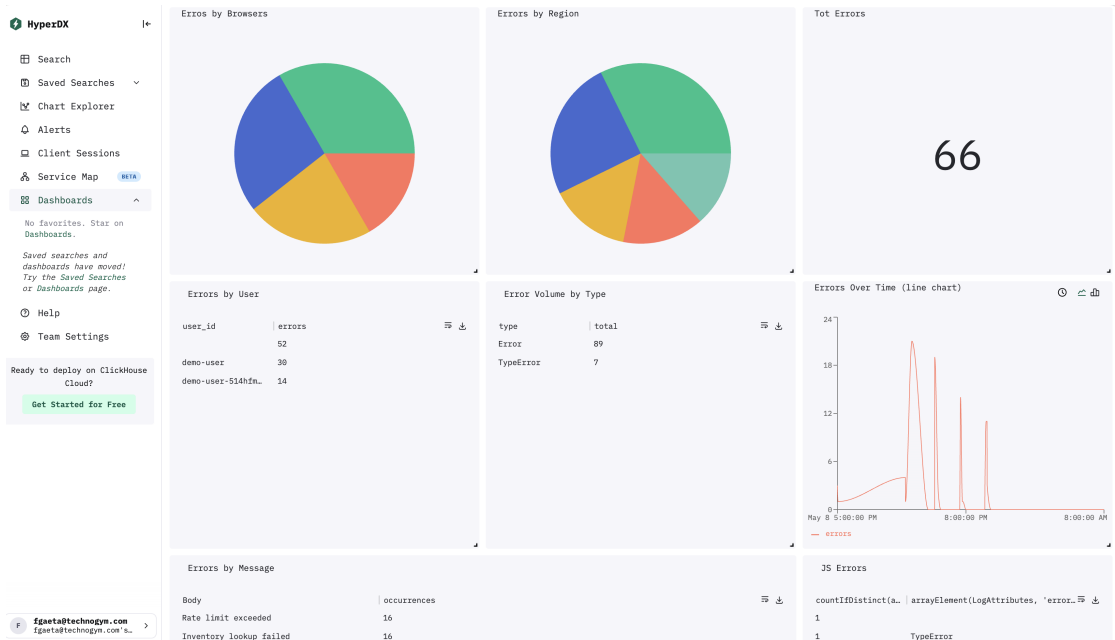


Figure 4.3: Dashboards in HyperDX to Track Errors

```
SELECT
  Body,
  count(*) AS occurrences
FROM otel_logs
WHERE
  lower(SeverityText) = 'error'
GROUP BY Body
ORDER BY occurrences DESC
LIMIT 10
```

Figure 4.4: SQL for Errors by Message

4.2 Development standards for observability integration in the source code

The focal points concern the implementation of a new *user story* dedicated to observability and the evolution of the modular architecture based on private pack-

ages.

User stories represent specific units of work that need to be completed, such as developing a new feature, implementing tracking infrastructure, or fixing a bug. They are used to planning and scheduling what modifications will be included in upcoming product releases and can be tracked thanks to a platform called Jira. The team has established strict conventions to link these user stories directly to their Git workflow:

- **Branch Naming Convention.** When a developer creates a new branch for a single user story, the branch name must begin with the Jira user story ID, followed by an underscore and a brief description of the task. In our case, the specific ID was “PROS-9099” (written as: *PROS-9099_Observability*).
- **Commit Message Convention.** The Jira issue ID must also be included inside the commit messages, typically placed in the footer section under “refs”. By putting the ID in both the branch name and the commit message, the team can easily trace all code modifications back to the specific Jira issue they were intended to solve. This is particularly useful because when a task is too large, it is typically broken down into multiple user stories instead of putting hundreds of IDs in the branch name. The team creates a single general branch, and by keeping the ID in the refs, they ensure the reference to the exact user story while underlining that all together aim to pursue the same objective.

The development model is rigorously structured around a standardized branching model to guarantee the stability of production environments and flexibility for releases. The basic rule establishes that each new feature or correction should start from the branch *main* (production source code). This strategy allows isolating changes and bringing them to production singularly. This is done to avoid the inclusion of non-tested functionalities in the release branch.

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

Branch Type	Description	Naming Convention
Main	Source code in production	main
Dev / Test	development and inspection environments	dev, test, alfa, beta
Feature	New functionalities (with Jira ID)	feature/ JiraID_BriefDescription
Release	Preparation for a new release	release/<name>

Table 4.1: Git Branch Naming Conventions

Also, commits need to follow a semantic standard to facilitate traceability on Jira. The structure is `<type>(<scope>): <description>`, where types are drawn from the conventional commits list (feat, fix, docs, style, refactor...), the scope has been decided internally to indicate the part of the application that has been modified (home, planner, training...), in the footer we find the references field for the Jira issues.

The application follows a modular philosophy, where components are interchangeable to reduce the dependency from specific libraries and to facilitate updates. The code is organized into reusable packages hosted in a private registry. The two main packages are “Infrastructure” with the basic logic, and “Web App UI” to manage UI components. The access to those requires authentication via private token locally configured in the NPM registry. The architecture is based on generic interfaces. For instance, the “Tracking Service” (initially referred to Google Analytics) is extended to the new Telemetry Service. The aim is not making the interface dependent on third-party libraries such as OTel, but these should be confined exclusively in the specific provider implementation (in our cases, ClickStack or Datadog). To achieve this, we defined a generic *TelemetryService* interface. Instead of directly importing from OpenTelemetry API, we created custom generic types, such as a dictionary for attributes, where the key is a string and the value can be a string, number, or boolean, so the interface itself has zero dependency. This strict separation guarantees that if the team decides to stop using OpenTelemetry in the future, the core application interfaces will not need to

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

be rewritten. The actual OTel logic is isolated within two specific implementation files, one regarding ClickStack for initial tests while waiting for Datadog’s platform activation and exploratory sessions, and the other for using the latter. The necessary OTel packages are installed in the package.json as standard dependencies rather than development dependencies, ensuring that they are included in the final compile application build.

4.2.1 Page views tracking, sanitization and session replay towards GDPR compliance

Once the telemetry abstraction layer has been introduced, the next step consisted in instrumenting the application to collect navigation events. Sending metrics, logs and traces was quite trivial, since we just had to import the code we had prefabricated in the demo app and apply some refinements to integrate it with the existing components. Another observability feature implemented through the new architecture was page views tracking, again taking inspiration to the already done work. Page views are essential telemetry signals to jump into user navigation flows and to allow the construction of usage analytics dashboards. To reach this goal, the activity focused on automatically tracking route changes throughout the application.

The tracking logic was centralized: instead of embedding telemetry calls inside individual pages which would cause *scattering* (literally spreading things over a wide area, hence calls towards tracking systems would be lost in the whole business logic, making each modification onerous and error-prone), the implementation leveraged the telemetry service abstraction and centralized page view registration within the routing layer. This approach reduced code duplication and guaranteed consistent event generation across the entire application. The *trackPageView* method uses the parameter “path” instead of the complete URL. This must be brought to attention because while the URL (composed of *window.location.href*) might include protocols and hosts, the *path* identifies the logical internal path (e.g. */people/contacts*). This choice was made for privacy protection to avoid the

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

unintentional capture of PII (Personal Identifiable Information) or sensible query parameters. What we did is called *sanitization*.

Listing 4.1: sanitization function

```
1  export function sanitizePath(path: string): string {
2  return path
3  .split('/')
4  .map(segment => {
5      // Check if segment is a GUID (with or without dashes)
6      // Pattern: 8-4-4-4-12 hex digits or 32 hex digits without dashes
7      const isGuid = /^[0-9a-f]{8}-?[0-9a-f]{4}-?[0-9a-f]{4}-?[0-9a-f]{4}-?[0-9a-f]{12}$/i.test(segment)
8
9      // Check if segment is a MongoDB ObjectId
10     // Pattern: 24 hexadecimal characters
11     const isObjectId = /^[0-9a-f]{24}$/i.test(segment)
12
13     // Check if segment is purely numeric (IDs like 123, 456)
14     const isNumericId = /^\d+$/i.test(segment)
15
16     // Replace GUIDs, ObjectIds, and numeric IDs with :id
17     if (isGuid || isObjectId || isNumericId) {
18         return ':id'
19     }
20
21     return segment
22 })
23 .join('/')
```

Sanitization links to the concept of data cleanliness, too. Sending row data to monitoring systems is not only risky in privacy terms, but could additionally cause a technical issue known as “cardinality explosion”. If dashboards receive tons of univocal entries, their statistical aggregation becomes unfeasible, making data unavailable for operational monitoring. The usage of a GUID (Globally Unique Identifier) composed of 32 characters by default separated by dashes in URL paths prevents metrics’ grouping. It means that every access to a specific resource would be seen as a different page because the user has changed. Sanitization is the only way which allows operators to visualize traffic as aggregated per sections. For instance, `/contacts/UUID-123` turns into more generic `/contacts/ID`. In the code we can find a sanitization function in the telemetry utilities which exploits regex to identify not only GUIDs, but also numerical and more compact “object IDs”

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

typically returned by databases. These identifiers get replaced with a generic “ID” which guarantees cleanliness across sent data.

This procedure is a must to comply with the **GDPR!** and other privacy regulations that end up being very impactful especially in the European context. The GDPR governs the collection of personal data within the EU territory. Its objective is defining what sensitive data is, how it can be collected, how to deal with biometric data which is another subset of the wider sensitive information group, and how to protect the privacy of individuals. In art.4, we encounter the definition of *personal data* as any information relating to a natural person, also known as data subject, identified or identifiable, directly or indirectly, by reference to an identifier such as name, ID number, location data, or more factors specific to the physical, physiological, cultural (...) identity of that person [?]. Enterprises must pay attention to the *principle of data minimization*, which states that personal data must be adequate, relevant and limited to what is necessary in relation to the purposes for which they are processed. In our case, it means we should not send any PII to the telemetry system, and we should avoid sending any sensitive data that could be used to identify a user.

Since we wanted to track user navigation flows and revise them in session replays, it was mandatory to implement a masking strategy to hide sensitive information, which leaded us to obscuring the input fields for usernames and passwords, address, telephone number, birthdate (...). Consequently, we also needed to reach the same results for dropdown menus used to select sensitive information, such as the birthdate through the common date picker interface. In the original code before bringing the new edits, Mouseflow was the tool used for user interactions granular monitoring. The system utilized the `SensitiveInfoClassName` constant (defined as `sensitive_info`) and the boolean `sensitive` prop to mark components that need to be hidden. Among these elements, profile pictures are also included, considering that platforms are generally not able to autonomously understand which images need to be covered as sensitive content. Mouseflow also prevented data capture at the source, masking information directly on the client-side to avoid any risky transmission, considering that post-processing would be actually forbidden

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

according to the company’s Zero-Trust Data Egress policy. Datadog RUM, instead, offers advanced instruments for privacy management, but the configuration must be calibrated in order to reflect the severity of our current system. Datadog provides two approaches for marking sensitive content: HTML attributes (`data-dd-privacy=“value”`) or CSS classes (`dd-privacy-value`). The available privacy levels are:

- **hidden:** the content of the element is not captured in the session replay, and it is replaced with a placeholder such as a gray rectangle, which is the safest solution since it does not even let third parties know the number of characters and words in the field;
- **mask:** the content of the element is captured, but it is masked with dots or asterisks, which means the length of the content is visible, except for the actual characters, rendering the wireframe. It is the best compromise between privacy and usability, but we could privilege “hidden” because there is nothing to be gained from knowing the length of a password or a username for our debugging purposes;
- **mask-user-input:** the content is captured, but the user input is masked with dots/asterisks while the rest of the content is visible, so that we can only obscure what is sensitive.
- **allow:** the content is simply captured and displayed in the session replay.

Our final strategy was adopting the CSS class approach, maintaining the existing `sensitive_info` class to ensure backward compatibility. This allowed all tags already marked with the `SensitiveInfoClassName` to automatically benefit from Datadog’s privacy masking without requiring modifications to hundreds of component instances. Subsequently, by navigating different parts of the app and reviewing the session replays, we continued adding the class where needed but where there was no previous reference to the sensitive information props. The default privacy behavior in Datadog initialization was set to ‘allow’, because it would have been significantly more complex to explicitly allow the hundreds of

non-sensitive elements, compared to selectively masking the dozens of sensitive components, the majority of which were already identified.

4.2.2 Performance monitoring: legacy and new APIs and Core Web Vitals

Performance monitoring concerns the systemic process of tracking, measuring, and analyzing key performance indicators to ensure goals are being met, and evaluate the progress and effectiveness of an application in this specific case [?]. The guidelines for the project were to keep **CWV**!s, page and resource loading time and React component rendering performance under control, besides analyzing API call response time.

API!s (**API**!s) [?] are the invisible backbone of modern software development. They enable applications and systems to communicate and share data efficiently with seamless interactions among platforms, services and applications. The API takes a request, sends it to the server, retrieves the data and then returns the response.

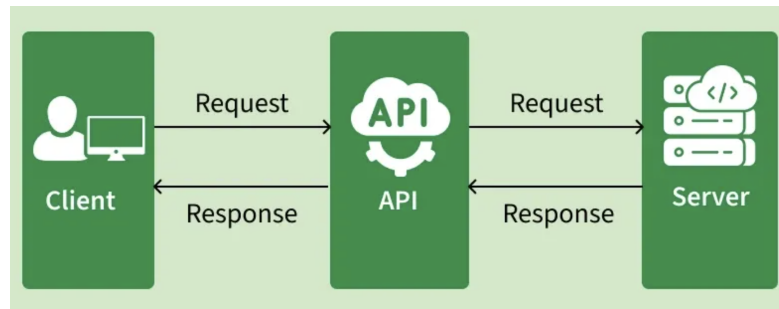


Figure 4.5: API Request-Response Cycle.

Source: <https://www.geeksforgeeks.org/software-testing/what-is-an-api>

There are numerous types of APIs we could mention, but in our case study what is relevant is understanding the *RESTful APIs*, which are based on the

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

principles of Representational State Transfer (REST). RESTful APIs use standard HTTP methods (GET to retrieve a record, POST to create a new one, PUT to update it, PATCH as a partial update, DELETE to remove the record) to perform operations on resources identified by URLs. They are stateless, meaning each request from a client to a server must contain all the information needed to understand and process the request. RESTful APIs are widely used due to their simplicity, scalability, and compatibility with web technologies. In our context, it is essential to understand the error categories that can be returned by the server in response to a request. These are represented by three-digit HTTP status codes that start from 4xx to 5xx. For instance, some of the most common mistakes that can be experienced by normal users are error 404 (Not Found), error 403 (Forbidden), error 401 (Unauthorized) and error 500 (Internal Server Error) or 503 (Service Unavailable). The 5xx category is the most dangerous because it indicates a server-side problem that prevents the request from being fulfilled, while the others concern client-side criticalities. Furthermore, we can encounter network errors (timeouts, connection refused), or custom business logic errors, which return a 200 status code (OK) even when the request is not successful, and the error payload attached should be investigated to verify if there is something wrong and where.

The latter is particularly crucial in our architecture. The old legacy APIs are managed with an explicit comment at code-level to check if there is an "error" array despite the 200 OK response. This was a common practice to separate HTTP errors from applicative errors, to distinguish when a request fails from when it arrives correctly, but the operation failed for business reasons. The new APIs, instead, are designed to return a 4xx or 5xx status code when an error occurs, following the standard HTTP conventions. This change improves the clarity of error handling and allows for more efficient debugging and monitoring of API interactions; as a matter of fact, the following steps involve a transition to the new ones. The migration of API legacy is a tactic necessity dictated by the **EoS!** (**EoS!**) of current servers, estimated for the beginning of next year (2027). There are two possible "versioning APIs" strategies teams can choose between, but right now they are adopting V1 which is a more trivial redirect, because with respect to the V2 strategy it is faster thanks to the fact that there is no need to re-create the API

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

from scratch, but the input and output will stay the same so that the test phase is faced quickly, the impact on the client is null since touchpoints are not affected and the transition is transparent. The result is a new infrastructure maintaining the legacy code, while in V2 a new paradigm with a new code would be introduced. The V2 strategy is still in the to-do list, but it is a long-term strategy that requires time, effort and risk for mistakes, therefore it is essential for now to minimize the impact on the client and accelerate parity tests.

Listing 4.2: API error handling

```
1 import HttpResponseMessage from '../..../http/models/HttpResponseMessage';
2 import HttpStatusCodeType from '../..../http/models/HttpStatusCodeType';
3 import RepositoryResponseError from '../RepositoryResponseError';
4
5 declare class RepositoryResponse<TData> {
6     data?: TData;
7     errors?: RepositoryResponseError[];
8     token?: string;
9     tokenNotValid: boolean;
10    permissionDenied: boolean;
11    mfaRequired: boolean;
12    statusCode?: HttpStatusCodeType;
13    constructor(
14        httpResponse?: HttpResponseMessage,
15        responseIsInBody?: boolean,
16        responseMapper?: (data: any) => TData
17    );
18    get hasErrors(): boolean;
19    get errorMessage(): string;
20    private getGenericError;
21 }
22
23 export default RepositoryResponse;
```

The **BE!** (**BE!**) teams - managing API calls, connections to databases, AI integration to work with data - were already exploiting Datadog in four main areas: **APM!** for all microservices, infrastructure and cloud monitoring to collect metrics from the underlying AWS infrastructure, alerting for critical reliability signals, logs and async processing with archive search for historical investigation. When **BE!** and **FE!** (**FE!**) are monitored in isolation, the views for each are disconnected even if they concern the same user journey. The FE lives through RUM and the BE through APM, and this creates a gap. By reaching the goal of linking RUM sessions

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

to backend traces, we get an end-to-end view. The correlation works through a distributed tracing context injection so that the RUM SDK ingests trace headers into every HTTP request to the backend.

Listing 4.3: RUM SDK trace context

```
1 datadogRum.init({
2   applicationId: '<APPLICATION_ID>',
3   clientToken: '<CLIENT_TOKEN>',
4   site: 'datadoghq.com',
5   service: 'professional-next',
6   allowedTracingUrls: [
7     "https://services.mywellness.com",
8     /^https:\/\/[^\/]+\..mywellness\.com\/
9   ],
10  trackResources: true,
11  trackLongTasks: true,
12  trackUserInteractions: true,
13 });
```

To show the chain, we need RUM configured with *allowedTracingUrls* to know which calls to correlate, and W3C trace context propagation *traceparent* to link the BE trace to its child. **CORS!** (**CORS!**) should already be configured to allow such headers, because the backend has to accept them due to security rules. Indeed, **CORS!** [?] is a mechanism for integrating applications by defining a way for client web applications that are loaded in one domain to interact with resources in a different domain. This is useful because complex applications often reference third-party APIs and resources in their client-side code. This is needed because any kind of discomfort that the client might experience while using the CRM Mywellness application is perceived as an interface/application error, even when it is not a responsibility of frontend programmers, but the common user has no technical expertise to understand it, hence their feedbacks will sound generic, too. Without this proper traceability methodology, a lot of time would be wasted looking for the source of the problem almost randomly checking code files.

Another important means to analyze performance and resource speed, quality and general effectiveness are **CWV!**s. **CWV!**s is a set of metrics that measure real-world user experience for loading performance, interactivity, and visual stability of the page [?]. The categories we are interested in refer mainly to the user

experience impact:

- **LCP (Largest Contentful Paint.)** It tracks how fast the main content loads, which can correlate to retention rate, because if the users abandon slow sites retention is directly affected negatively.
- **FID/INP (First Input Delay/Interaction to Next Paint.)** It is about responsiveness of the elements. At the beginning there was only FID, which measured only the time of the first user input to start the interaction, this is why nowadays is often sided or replaced by INP, which is about the total response time that comprehends all interactions, that can be more useful in application contexts.
- **CLS (Cumulative Layout Shift).** It concerns visual stability. If there are unexpected layout shifts, it can lead to user errors or general poor **UX!**, which discourages people to use the app.
-
- **FCP (First Contentful Paint).** It measures how fast the page shows the first visible content. It is not a core web vital, but it is still an important web performance metric, and Google still reports it to help diagnose loading performance.

To qualify as "good" performance, Google establishes the following thresholds: LCP should occur within 2.5 seconds, INP should be below 200 milliseconds, and CLS should remain under 0.1. These targets must be met for at least 75% of page visits. For Technogym, establishing these benchmarks as **SLI!** allows the team to define clear **SLO!** and trigger alerts when performance degrades, enabling proactive intervention before users experience significant impact. Integrating CWV monitoring into the observability infrastructure provides real-time visibility into user experience across different devices, network conditions, and geographic regions. Through OpenTelemetry's browser instrumentation, these metrics can be correlated with backend traces, allowing the team to identify whether a poor LCP

4.2. DEVELOPMENT STANDARDS FOR OBSERVABILITY INTEGRATION IN THE SOURCE CODE

stems from slow API responses, excessive JavaScript execution, or network latency. This end-to-end visibility transforms CWV from isolated frontend metrics into actionable insights that guide both frontend and backend optimization efforts.

Chapter 5

Establishing a single pane of glass: error, performance, and analytics monitoring through dashboards in Datadog

In the previous chapter, we introduced the main expedients to depict the baseline to grasp a detailed understanding of the application's performance independently of the platform adopted. In this final chapter, instead, we are about to dissect the technical procedures for building dashboards in Datadog to facilitate a comprehensive and fast analysis for developers.

A dutiful clarification to make is that the majority of the code exposed in the fourth chapter was needed specifically to support ClickStack's data collection and HyperDX implementation. This is indeed the main factor which led to selecting Datadog as first choice: ClickStack demonstrated to be for sure a great frontier, but is visibly a work in progress and still needs adjustments to make life effectively easier, while Datadog can flatter years of existence that allowed it to become an undefeatable standard today concerning monitoring and analysis. Practically speaking, this means that the afore-mentioned platform already em-

beds the logic to configure sensible data masking with the possibility to choose among various possibilities without needing a heavy and supplementary sanitization logic in the application code, plus it tracks automatically page views because it is capable of catching every URL path change if the functionality is enabled, so that part of code becomes skippable, too. Another advantage is that, thanks to initialization commands such as *“trackUserInteractions”*, *“trackLongTasks”* and *“trackResources”*, Datadog can automatically track every click, page navigation, frustration signal, error, without requiring additional instrumentation. This reduces the amount of manual coding required for telemetry collection and ensures that critical performance metrics are captured consistently across the application. Another simplification was the automatic registration of many global attributes that, in the case of ClickStack, needed to be manually set up, like the browser name or version, the operating system or device type used by the customer, the application version, geographical information, and so on.

5.1 Error monitoring dashboards and alerting configuration

Throughout the process, it has been decided that the priority for the enterprise was to intercept errors with severity labeled as “critical” and “high”, because it is the biggest pain point for the company in terms of customer care and application quality.

As already mentioned in some old sections, the aim was to take care of handled errors (API errors and business logic, browser-side infrastructural errors, device and hardware errors) and unhandled errors that the code does not predict and does not capture through try-catch (React Rendering, uncaught promise rejections, JavaScript errors). These unhandled are automatically recognized by the browser and the Datadog SDK intercepts them without human intervention, so that developers do not lose error information. Handled errors are called this way because they foresee the faults and they are handled somehow (e.g., by showing

5.1. ERROR MONITORING DASHBOARDS AND ALERTING CONFIGURATION

an error message to the user, or with a retry). The distinction between handled and unhandled is fundamental to understand the implications of each, because the former represents a UI degradation but keeping the app functioning, they might go unnoticed, they require manual tracking and are managed in the code through the “`recordException()`” method adding a try-catch block to avoid crashes and explicitly defining the severity level chosen among low, medium, high and critical and eventually adding attributes to better understand the context during debugging; while the latter often make the app crash, are visible (there is a white screen or the functionality does not respond), are auto-captured and to recover the page must be reloaded.

Listing 5.1: `recordException()` method for handled errors

```
1  recordException(error: Error, severity?: ErrorSeverity, attributes?:
   Attributes): void {
2      // Always add severity to attributes (default to 'high' if not specified)
3      const enrichedAttributes = {
4          ...attributes,
5          severity: severity || 'high'
6      }
7
8      this.recordError({
9          error,
10         level: 'ERROR', // Technical log level is always ERROR
11         attributes: enrichedAttributes
12     })
13 }
```

Moreover, React introduces a third, peculiar category which are React Rendering errors: when a component fails to render, it can cause the entire React tree to unmount, which means that the interface disappears and the user sees a blank page, making the application unusable. This is problematic in a web-app where the crash of a separate section should not be able to affect and impede the access to core functionalities. To control these situations, the solution is introducing an Error Boundary, comprehending special components that catch errors in their “subtree” and display a fallback UI instead of crashing the whole app. The specific implementation of our ErrorBoundary does not only show a fallback, but enriches the context of the error:

Listing 5.2: `componentDidCatch()` implementation for ErrorBoundary

5.1. ERROR MONITORING DASHBOARDS AND ALERTING CONFIGURATION

```
1 componentDidCatch(error: Error, errorInfo: ErrorInfo): void {
2   // Create a new error with component stack as recommended by Datadog
3   // This makes the component tree visible as the main stack trace in Datadog
4   RUM
5   const renderingError = new Error(error.message)
6   renderingError.name = 'ReactRenderingError'
7   renderingError.stack = errorInfo.componentStack
8   ;(renderingError as any).cause = error
9
10  // Track the error with React-specific context and any additional attributes
11  App.telemetry?.recordException(renderingError, 'high', {
12    'error.type': 'react_error_boundary',
13    'error.source': 'ErrorBoundary.componentDidCatch',
14    ...this.props.attributes
15  })
```

Instead of just tracking the error, we rebuild it. This is done because the JavaScript stack trace of “error” explains where in the code the error has been generated (in which file, at which line), but in a complex React application it is often not enough to quickly find it, therefore is it better to have the component tree to recreate the path of the functions and parameters involved in a file. Furthermore, chaining the original error into “cause” avoids information loss, since both the component stack and the original JS stack are kept in Datadog logs.

Another important category of errors to manage were related to APIs. Since these are called innumerous times inside the code, it was not clever to put up a “distributed tracking” pattern, that is repeating the try-catch block structure adding severity and context-specific attributes to every part of the code where we needed to monitor API and business logic errors, which would have resulted in tens of files containing the same code structure with just some slight variations. It could lead to duplication (the same error is tracked twice or more), inconsistency if more developers were working on different parts of the code tracking the same typology of errors with different attributes structure, and false positives due to blindly tracking business logic mistakes as if they were technical errors (for instance, if a login fails, it is not necessarily due to a malfunctioning in the connection between app and database, but maybe the user has just inserted the wrong credentials, and it does not make sense to recollect this huge part of expected data). Plus,

5.1. ERROR MONITORING DASHBOARDS AND ALERTING CONFIGURATION

maintenance would have been a nightmare.

Consequently, the strategic solution was centralizing the API error tracking into the HTTP client or repository layer in the application's infrastructure. If we track in a single point where all API calls go through, we end up with a single source of truth where we define what is and is not an error, a consistent system because all API errors will contain the same basic attributes, and light maintenance and easy evolution. The try-catch block stays only for application management for recovery and UI feedback, but actual tracking is delegated to an inferior layer.

In this context, another important role is played by the *beforeSend* hook, a function invoked by Datadog before sending any event to the cloud. It works as a firewall that inspects, modifies, enriches or blocks the event depending on the conditions. It is crucial because Datadog RUM captures everything, and this is a powerful approach because it allows us to filter later in the dashboards so that we are sure not to lose any information, but the cost is higher noise. Sometimes, we already know that some events do not constitute a real error, so we filter them in advance with the *beforeSend* to keep only relevant signals. In our case study, a typical example is a false positive error such as the MFA: when the users utilize a multifactor authentication, after inserting the password to perform the login they are not directly redirected to the home page, because they must complete the intermediate step of writing down the security code, and this resembles an error with status code 401, but it would be useless noise to collect all these situations, so the *beforeSend* already excludes the MFA request by the error cases:

Listing 5.3: snippet of the *beforeSend* hook

```
1  beforeSend: event => {
2      // MFA code required (401/403 with MFA context). NOT AN ERROR, skip
      tracking
3      // This is expected behavior when user needs to enter MFA code
4      if (
5          (statusCode === 401 || statusCode === 403) &&
6          (msg.includes('MFA') ||
7           msg.includes('mfa') ||
8           msg.includes('two-factor') ||
9           msg.includes('2FA') ||
10          event.view?.url?.includes('/auth/login'))
11      ) {
```

5.1. ERROR MONITORING DASHBOARDS AND ALERTING CONFIGURATION

```
12     return false // Drop event, not a real error
13 }
```

The same goes for network abort type of errors and other 4xx errors that we already recognized as false positives. The reason why we put them in the `beforeSend` rather than avoiding their capture is that, foremost, the platform catches them automatically because they do not pass through the `recordException()` method, so we cannot exert any control without the hook, plus it would still be fundamental to collect whatever signal especially for building performance-related dashboards. It does not matter if some API calls are not actual errors, we still need to use that information to check the page loading time, the endpoint speed and other analytical parameters.

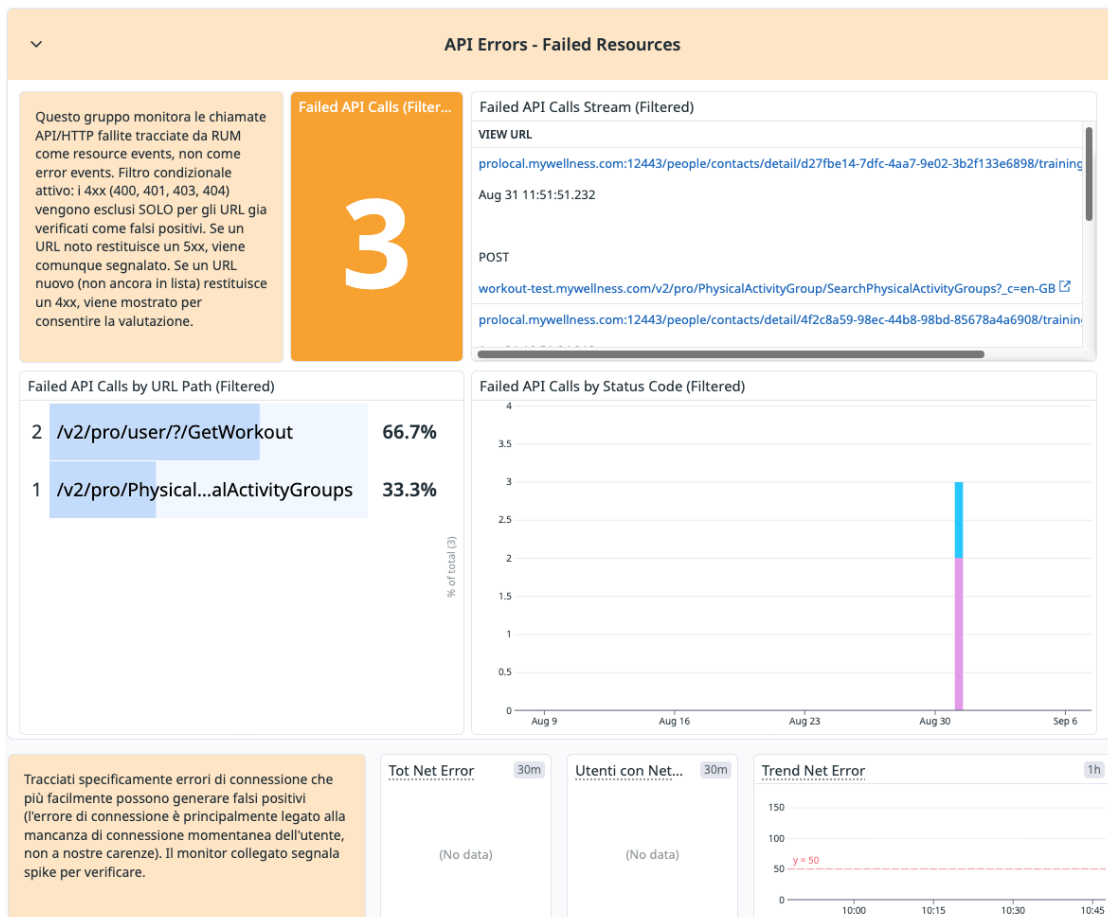


Figure 5.1: API error monitoring specific overview

Thanks to the `beforeSend()`, we could also assign a default severity to all those auto-captured errors that do not naturally come with it. For example, we could establish that some errors related to `WebSocket/SignalR` were harmless almost always, so we opted for a low severity level with the intention of just checking every time if they are unnoticeable. The severity pattern we applied for anything else unspecified and unknown defaults to “medium” as a compromise.

The criteria to assign a specific severity deserve attention, too. Generally, three dimensions must be considered:

- how severely it impacts the user experience. If the user cannot work anymore, it is critical/high, if the degradation is visible for the user but does not really block any workflow it is medium, if the user might not even realize the error it is low.
- the scope. The severity escalates depending on how many users get penalized, how many functionalities are impacted and which ones, if the error is isolated or starts propagating.
- the level of business criticality. If it affects revenues for payments and billing, or risks concern compliance it is critical/high. Severity escalates then depending on the reputational damage (e.g., if it appears in a demo, if for all public clients...)

The difference between high and critical usually concerns the importance and usage rate of the core functionality involved. In a critical situation, the core functionality is utterly compromised, while high severity is still significant, but a workaround may exist and the functionality is not totally blocked.

5.1. ERROR MONITORING DASHBOARDS AND ALERTING CONFIGURATION

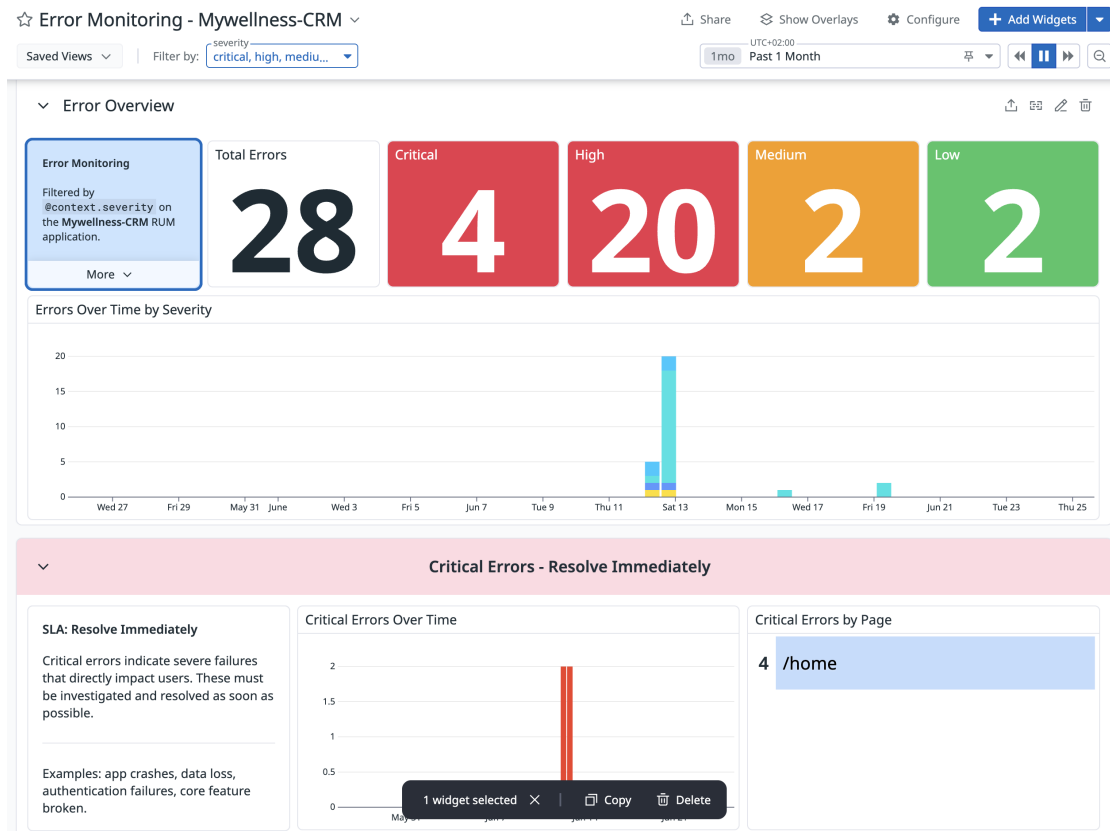


Figure 5.2: Error monitoring overview filtered by severity

Dashboards are used for passive monitoring, but the alerting system embodies the proactive counterpart, because it is in charge of sending a notification when a certain pre-established threshold or condition gets overcome. Each monitor can also be tuned in order to send the notification to specific teams and people via predefined channels. The chosen channels are email and Slack, the two main media used for internal communication.

Surely, thresholds will vary depending on the severity of the error. For instance, a spike in API errors is quite severe, so it has been decided that a spike starts when the number of errors exceeds 5 within 10 minutes, but more specifically if the API errors come with status code 5xx, it gets signalled immediately with just a single error. This results to be very helpful because it lightens the load for developers who do not need to check the dashboard every time.

Another type of monitor adopts Datadog's anomaly detection algorithm, which automatically learns the normal behavior of a metric and triggers an alert when it detects a significant deviation from that baseline. If the error rate is typically 0.5% it sends a notification reaching a higher value like 1%, and this mechanism is useful to catch errors that do not fit into a specific threshold, and to identify problems before they dramatically escalate, and that users start complaining.

5.2 Dashboard migration from Google Analytics to Datadog

The final objective for concluding the observability project, after all this research and tests over platforms and data collection, is migrating all the data managed in Google Analytics to Datadog, which means rebuilding the whole dashboard suite in order to integrate everything in a single platform. Lots of example images will be provided in this section, but they will be taken primarily from the test environment and mocked local interactions in order to avoid the disclosure of any sensitive information.

Google Analytics serves as the industry standard for product and marketing analytics. Its internal logic is built upon an event-driven data model, where every user interaction is recorded as an independent event accompanied by custom parameters. The platform's widespread adoption stems from its ability to democratize data for non-technical stakeholders, since it does not require complex SQL or custom query languages which is a characteristic we can attribute to Datadog, too. By transitioning to Datadog, the perspective gets shifted from marketing analytics to technical observability. GA simply answers the question of what users are doing, while Datadog RUM is designed to understand the underlying reasons behind user behavior, their technical experience and why the system might have failed. The only great limitation of Datadog is it has a shorter retention period compared to GA, making monthly comparisons tougher, and being a paid platform it requires high cost scaling based on volume and custom metrics.

Acknowledgements

This thesis represents the end of one of the most beautiful, despite hard at times, journeys of my life, thanks to the new, fascinating things I have studied and the wonderful people I have met.

First of all, I am grateful to my supervisor, Professor Matteo Francia, not because it is a formality, but because he has truly been a great mentor, capable of explaining complex concepts while also having fun with his students at the end of his lectures, because he is both a great professor and person, he has eased our study path with his humor and, specifically to me, he has been a psychological support, too, helping me every time I had doubts about my next steps. A big thanks also to my co-supervisor, Davide Domini, who since the first year has been mainly a friend, he helped me to solve all the endless bugs in my terminal and always knew how to encourage me.

The most important thanks go again and always to my family. I know that, for you, I am the perfect daughter that can do whatever she can, but it is not true, and I have never felt this way towards myself. But I also know that if I am brave, ambitious and stubborn enough today to think I can face two years studying informatics to realize my childhood dream and survive six months of programming for an internship, it is because you raised me making me feel the love and safety I needed to be able to take risks, knowing I could have never disappointed you because you would have been always proud of me even committing the worst failure of my life, and because after all your sacrifices you deserved me to try hard, and I hope to give you my success as a gift every day.

The best feeling was finding colleagues that I have always called friends. From those who came into my life during the Bachelor's and never left, Alessia, Tommaso, Cinzia and Greta, to all my DTM fellows and the new people I met. You were able to make me discover new traits of my personality, new activities to enjoy, new objectives to pursue, you were always ready to support me even when I was unsure about everything and just overthinking my next ten years of life. As I already told you, I could endure other thousands of university courses with you, because you are the best medicine for my mind and my soul. It destroys me to think I won't see you every day anymore and that we are all going to be scattered around the world, but I know you are the kind of people that find a way to see each other and keep on with our adventures as we left the day before. Thank you for the esteem you show me, it is completely reciprocated, and I know you will be extremely successful. You will always feel like home and I deeply love you.